CONCEITOS FUNDAMENTAIS DE SISTEMAS CONCORRENTES

FPPD — Unidade 2

11 de Março, 2026

ROTEIRO

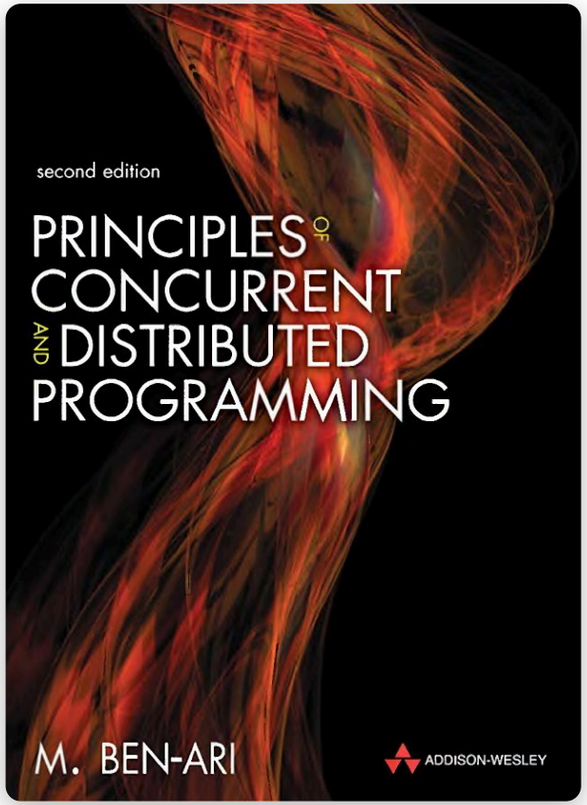
1 | Definições e terminologia de concorrência

2 | Cenários, estados e entrelaçamento

3 | Instruções atômicas e condição de corrida

4 | Seção crítica e mecanismos de prevenção

5 | Propriedades, justiça e problemas clássicos



O QUE É CONCORRÊNCIA?

DEMO

Um **programa concorrente** é um programa em que um conjunto de processos sequenciais é executado simultaneamente.

PROGRAMA SEQUENCIAL

- Declarações, atribuições, controle de fluxo
- Funções e módulos para organização
- Instruções executadas em **sequência**

PROGRAMA CONCORRENTE

- Múltiplos processos sequenciais
- Execução "ao mesmo tempo"
- Entrelaçamento de instruções
- Não-determinismo na execução

EXEMPLO EM GO

```go
package main

import (
    "fmt"
    "sync"
    "time"
)

var (
    mutex           sync.Mutex
    simulation_time int = 0
)

func DoWork(TaskId, iterations int) {
    for i := 1; i <= iterations; i++ {
```

NÃO-DETERMINISMO NA PRÁTICA

O **mesmo programa** executado duas vezes produz saídas **diferentes**:

EXECUÇÃO 1

```
0:      f1
1:            f1
2:                  f1
3:                        f1
4:      f2
5:            f2
6:                  f2
7:                        f2
```

EXECUÇÃO 2

```
0:            f1
1:                        f1
2:      f1
3:                  f1
4:                        f2
5:      f2
6:            f2
7:                  f2
```

A ordem de execução das goroutines muda a cada execução — o escalonador decide quem roda quando. Esse é o **não-determinismo** da concorrência.

DEFINIÇÕES FORMAIS DE CONCORRÊNCIA

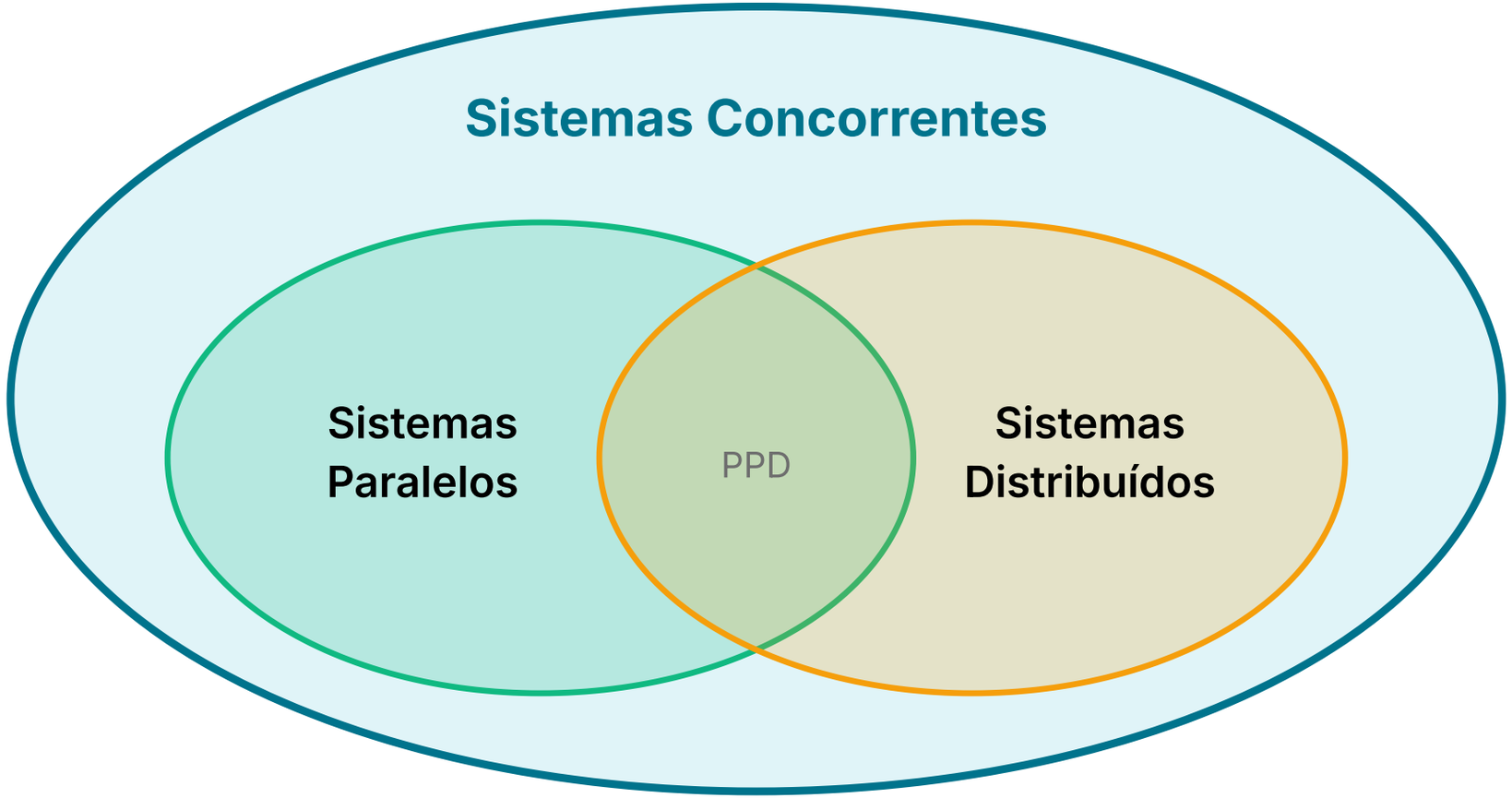
> *"Ability of different parts or units of a program, algorithm, or problem to be executed out-of-order or at the same time simultaneously, without affecting the final outcome."* — Leslie Lamport

> *"Concurrent computing is a form of computing in which several computations are executed concurrently — during overlapping time periods — instead of sequentially."* — Wikipedia

Ponto essencial: **concorrência não exige paralelismo físico** — pode ocorrer em um único processador via time-sharing.

SISTEMAS CONCORRENTES

Múltiplos processos executam de forma simultânea. Sistemas paralelos e distribuídos são subconjuntos dos sistemas concorrentes.



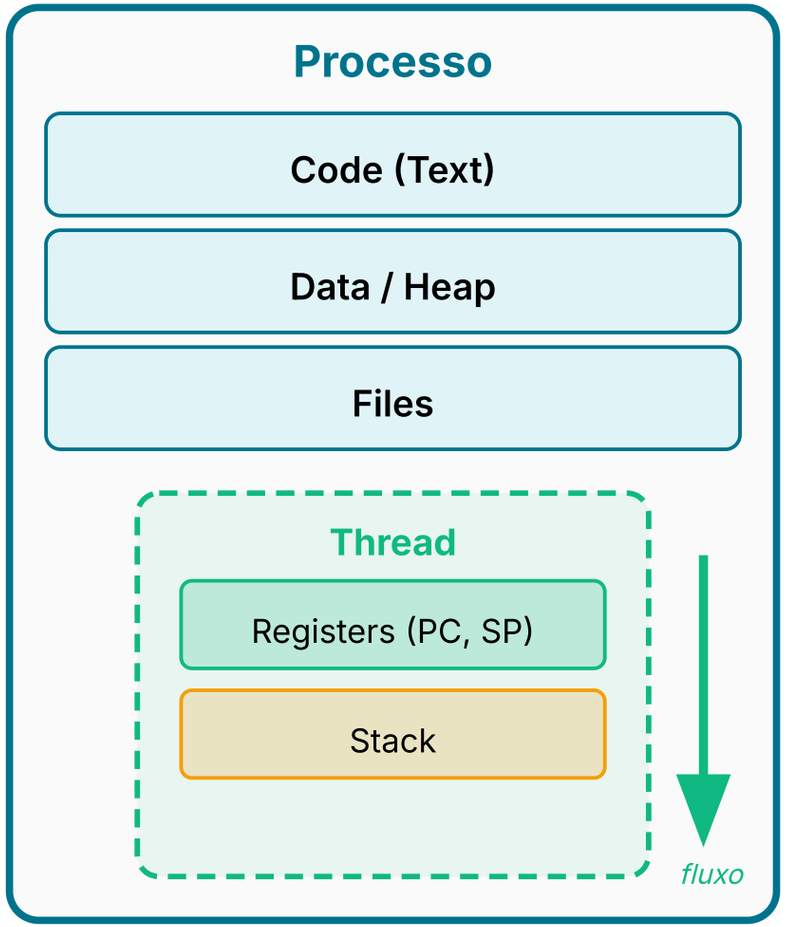
PROCESSOS

Em SO, um **processo** é uma instância de programa em execução. Cada processo contém:

- **Code/Text** — instruções a serem executadas
- **Data/Heap** — variáveis globais e dados dinâmicos
- **Stack** — variáveis locais e chamadas de função
- **Registradores** — estado de execução (PC, SP, etc.)
- Recursos: arquivos, sockets, etc.

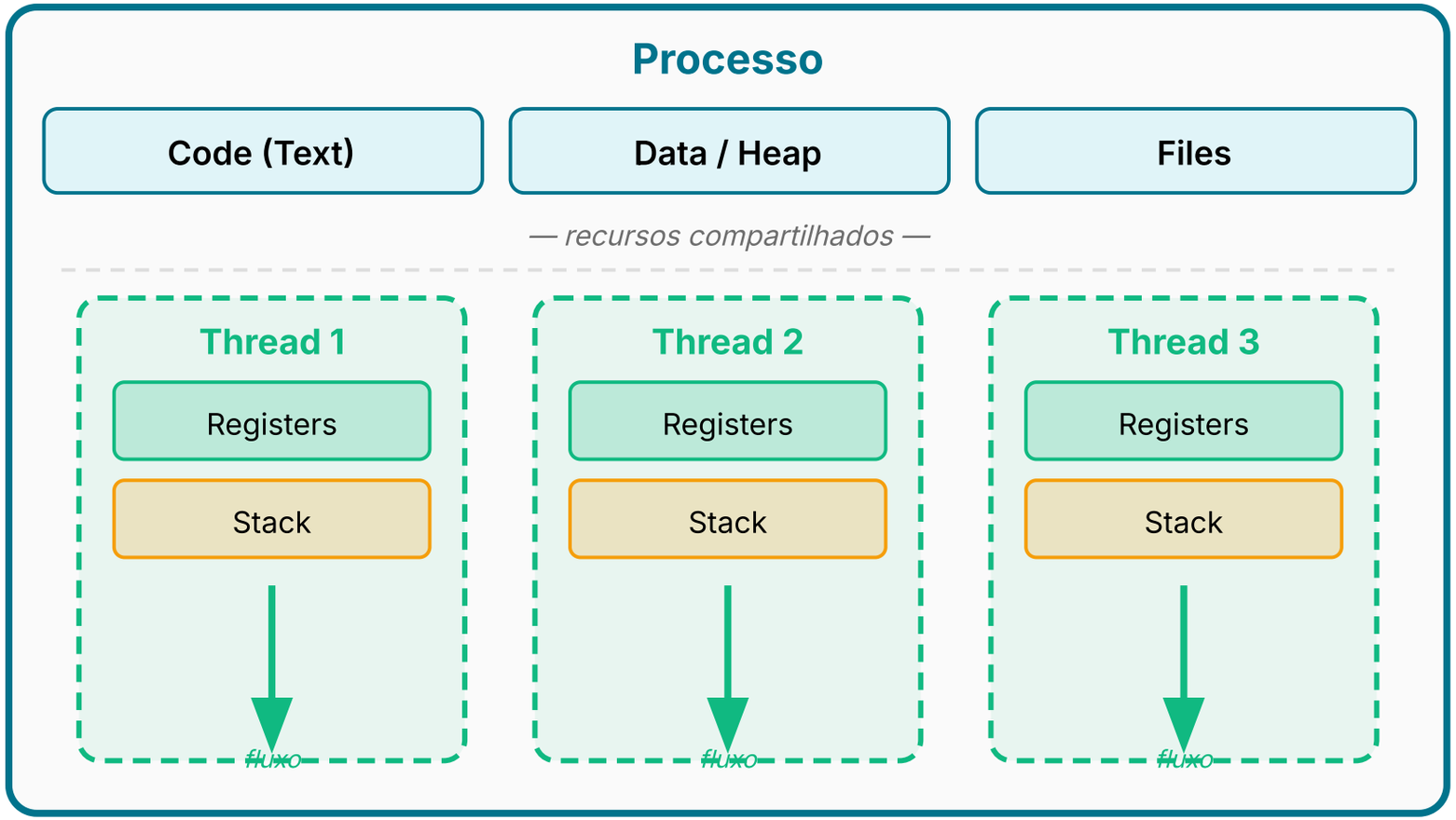
Todo processo tem pelo menos **um fluxo de execução** (thread).

Um processo pesado é um processo com uma única thread.



PROCESSOS COM MÚLTIPLAS THREADS

Cada thread possui PC, stack e registradores próprios. Todo o resto é **compartilhado**. Chamamos de **processos leves**.



PROCESSO PESADO VS. PROCESSOS LEVES

PROCESSO PESADO

- Uma única thread de execução
- Memória **isolada**
- Comunicação via IPC (pipes, sockets)

PROCESSOS LEVES (THREADS)

- Múltiplas threads no mesmo processo
- Memória **compartilhada**
- Comunicação direta via variáveis

A comunicação entre threads é muito mais rápida que IPC entre processos pesados, mas exige **sincronização cuidadosa**. Na disciplina, Go usa **goroutines**, que são ainda mais leves que threads do SO.

TERMINOLOGIA NA DISCIPLINA

PROCESSO CONCORRENTE

Código sequencial que roda de forma concorrente com outros processos.

No contexto de SO e linguagens: pode ser um processo pesado ou uma thread (processo leve).

PROGRAMA CONCORRENTE

Conjunto de processos que **colaboram** na resolução de um problema.

Comunicam, compartilham dados, precisam de sincronização.

Concorrência entre programas **diferentes** (ex.: várias aplicações na mesma máquina) não exige coordenação entre eles — é tratada pelo SO.

CONCORRÊNCIA LÓGICA E FÍSICA

CONCORRÊNCIA FÍSICA

Cada processo executa em um processador dedicado — paralelismo real.

Ex.: 4 processos em 4 cores

CONCORRÊNCIA LÓGICA

Multiplexação de um número **menor** de processadores entre os processos.

Ex.: 6 processos em 4 cores; caso extremo: time-sharing em 1 core

Os conceitos e técnicas que estudaremos valem para **ambos os tipos** de concorrência — a abstração é independente do hardware.

MOTIVAÇÃO PARA CONCORRÊNCIA

MODELAGEM NATURAL

- Muitos problemas são inerentemente concorrentes
- GUIs, servidores, jogos, sistemas em tempo real
- Linguagens modernas oferecem suporte nativo

DESEMPENHO

- Arquiteturas atuais são multicore
- Ganho de desempenho com paralelismo
- Aproveitamento do hardware moderno

ROBUSTEZ

- Isolamento de falhas
- Responsividade do sistema
- Tolerância a falhas

DESAFIOS DA PROGRAMAÇÃO CONCORRENTE

Dois desafios centrais:

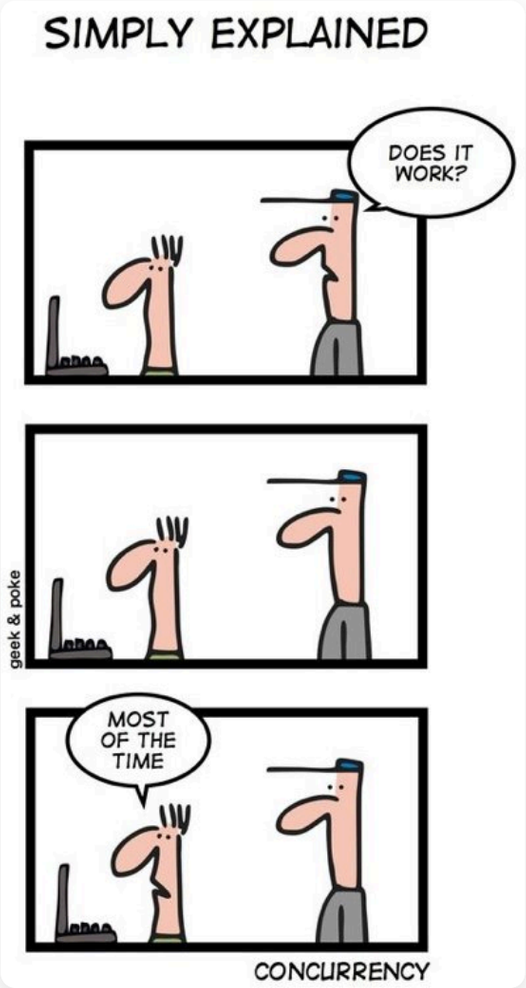
- **Comunicação** entre os processos
- **Sincronização** entre os processos

Se os processos pertencem ao mesmo programa, eles **colaboram** na resolução do problema — precisam comunicar e coordenar-se.

Por que é tão difícil?

- Implementar comunicação e sincronização de forma eficiente e segura é muito difícil
- Erros podem causar "congelamento" do sistema
- Depuração extremamente difícil

CONCORRÊNCIA — "SIMPLY EXPLAINED"



Fonte: Geek & Poke

EXEMPLO DE DESAFIO — CONDIÇÃO DE PARADA

DEMO

Perda do controle de fluxo dos processos: como saber quando todos terminaram?

COMO ESPERAR POR TODOS?

- Em alguns ambientes, quando o processo que **disparou** os outros termina, ele **mata** os restantes
- Ex.: em **Go**, quando a função `main` retorna, todas as goroutines são encerradas

COMO GARANTIR QUE TODOS TERMINARAM?

- Em outros ambientes, o programa não termina graciosamente (*gracefully*) enquanto todos os processos não terminam
- Ex.: em **MPI**, todos os processos precisam chamar `MPI_Finalize`

Em Go, usamos `sync.WaitGroup`, `time.Sleep` (temporário), ou canais para sincronizar o término. Veremos isso na prática nas próximas aulas.

ABSTRAÇÕES PARA CONCORRÊNCIA

Cenários, estados, transições e entrelaçamento

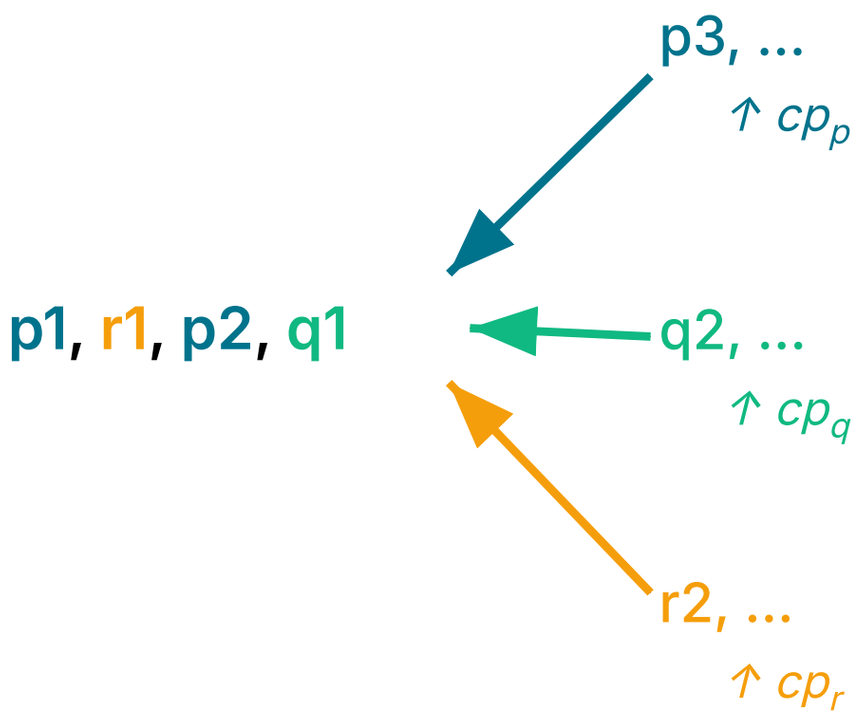
PROGRAMA CONCORRENTE — DEFINIÇÃO FORMAL

> Um programa concorrente consiste de um conjunto finito de **processos sequenciais**, escritos usando um conjunto finito de **instruções atômicas**.

- A execução se dá pelo **entrelaçamento arbitrário** das instruções atômicas dos processos
- Uma sequência de execução possível é chamada de **computação**
- Os diferentes resultados possíveis são chamados de **cenários**

CENÁRIOS E CONTROL POINTER

- A próxima instrução de cada processo é definida pelo seu **control pointer** (cp)
- Cada processo tem o seu cp
- Em cada passo, a próxima instrução é **escolhida** entre as instruções apontadas pelos cp dos processos



CENÁRIOS POSSÍVEIS

Dois processos (p e q), cada um com 2 instruções atômicas. Cenários válidos:

p1 → p2 → q1 → q2

p1 → q1 → p2 → q2

p1 → q1 → q2 → p2

q1 → p1 → q2 → p2

q1 → p1 → p2 → q2

q1 → q2 → p1 → p2

p2 → p1 → q1 → q2 não é válido — viola a ordem sequencial de p. A ordem **interna** de cada processo é sempre preservada.

EXPLOSÃO COMBINATÓRIA

O número de cenários é dado por $\mathbf{(n \cdot k)! \,/\, (k!)^n}$, onde *n* = processos e *k* = instruções por processo:

Processos (n)	Instruções (k)	Cenários
2	2	6
2	5	252
3	3	1.680
3	5	756.756
4	5	305.540.235

Com apenas 4 processos e 5 instruções cada, já temos **mais de 305 milhões** de cenários. Testar todos por execução é **impraticável** — por isso precisamos de verificação formal ou mecanismos de sincronização.

NOTAÇÃO INDEPENDENTE DE LINGUAGEM

Os conceitos de concorrência são universais — podemos representar programas concorrentes usando uma notação formal independente de linguagem (Ben-Ari):

Algoritmo 2.1: Programa concorrente trivial	
integer n ← 0	
p	**q**
integer k1 ← 1 p1: n ← k1	integer k2 ← 2 q1: n ← k2

ESTRUTURA DA NOTAÇÃO

1. **Título** do programa
2. **Variáveis globais** — compartilhadas entre todos os processos
3. **Colunas** — uma para cada processo do programa (p, q, r, ...)
4. Cada processo pode ter **variáveis locais** (privadas)
5. **Instruções atômicas** numeradas (p1, p2, ... / q1, q2, ...)

Dois cenários: **n = 1** ou **n = 2** — depende de quem executa por último.

ESTADOS E TRANSIÇÕES

O estado de um programa concorrente é uma tupla contendo: os valores do **cp de cada processo** e os valores de **todas as variáveis** (globais e locais).

VERSÃO SEQUENCIAL (ALGORITMO 2.2)

integer n ← 0, k1 ← 1, k2 ← 2

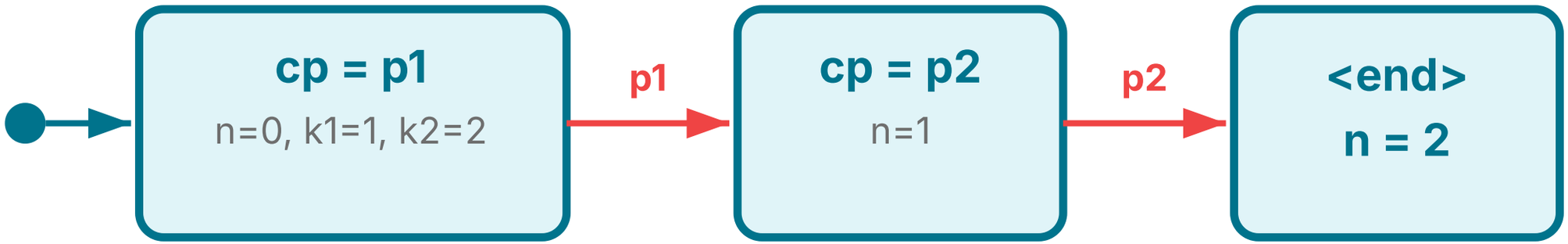
p1: n ← k1

p2: n ← k2

Um único cenário possível: resultado `n = 2`

ESTADOS E TRANSIÇÕES — DIAGRAMA

```
integer n←0, k1←1, k2←2
p1: n ← k1
p2: n ← k2
```



Cada nodo = estado (cp + variáveis) · Cada seta = transição (execução de uma instrução)

- Para cada estado existe **um único nodo** no diagrama
- Todos os estados são **alcançáveis** a partir do estado inicial
- Não deve haver ciclos (significariam computação infinita)

ESTADOS E TRANSIÇÕES — VERSÃO CONCORRENTE

Agora considere o mesmo programa com **dois processos concorrentes** (p e q):

ALGORITMO 2.1 — PROGRAMA CONCORRENTE TRIVIAL

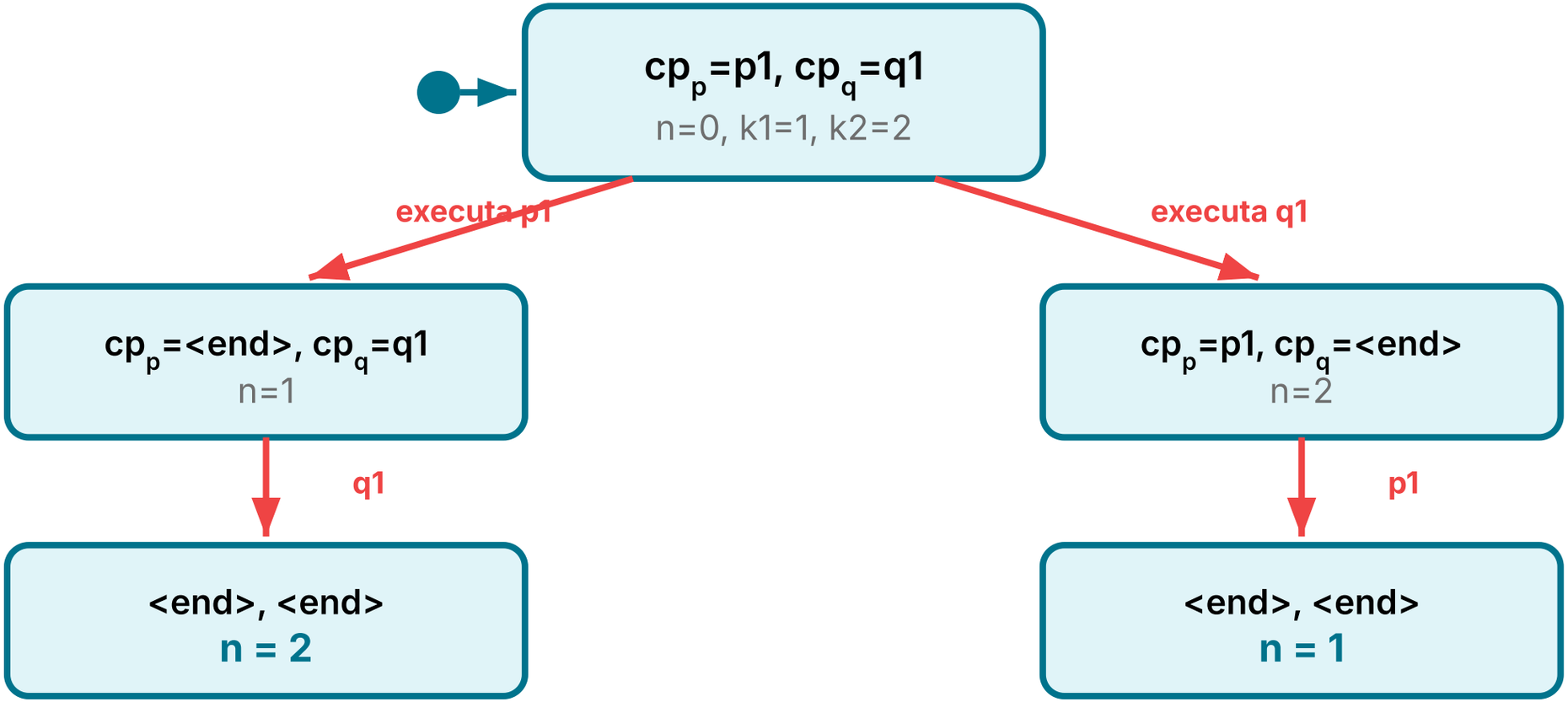
integer n ← 0	
p	**q**
integer k1 ← 1 p1: n ← k1	integer k2 ← 2 q1: n ← k2

O estado agora inclui o **cp de dois processos**. A partir do estado inicial, temos **duas transições possíveis** — gerando dois cenários diferentes.

ESTADOS E TRANSIÇÕES — PROGRAMA CONCORRENTE

integer n ← 0	
p	**q**
integer k1 ← 1 p1: n ← k1	integer k2 ← 2 q1: n ← k2

O mesmo algoritmo agora com dois processos concorrentes (p e q):



O estado inclui o cp de **todos** os processos. Duas transições possíveis do estado inicial geram dois

cenários com resultados diferentes: **n = 2** ou **n = 1**.

REPRESENTAÇÃO TABULAR

Diagramas de estados podem ficar complexos para programas grandes. Uma alternativa mais prática é a **representação tabular**.

Processo p	Processo q	n	k1	k2
p1: n ← k1	q1: n ← k2	0	1	2
(<end>)	**q1: n ← k2**	1	1	2
(<end>)	(<end>)	**2**	1	2

1. Para cada **estado** do cenário, temos uma **linha** na tabela.
2. A primeira linha representa o **estado inicial** do programa.
3. **Negrito** indica a instrução que foi **executada** naquele passo.
4. Se a instrução for uma atribuição, a **atualização** só aparece na **linha seguinte**.

REPRESENTAÇÃO TABULAR

Diagramas ficam complexos; a **representação tabular** é mais prática para cenários específicos:

Cenário 1: p1 → q1

Processo p	Processo q	n	k1	k2
p1: n ← k1	q1: n ← k2	0	1	2
(<end>)	**q1: n ← k2**	**1**	1	2
(<end>)	(<end>)	**2**	1	2

Cenário 2: q1 → p1

Processo p	Processo q	n	k1	k2
p1: n ← k1	**q1: n ← k2**	0	1	2
p1: n ← k1	(<end>)	**2**	1	2
(<end>)	(<end>)	**1**	1	2

Negrito indica a instrução executada. A atualização da variável aparece na **linha seguinte**.

ENTRELAÇAMENTO ARBITRÁRIO

Na abstração que usamos, analisamos **apenas** o entrelaçamento das instruções — **não** o tempo de execução.

- Não nos preocupamos com o **tempo** de execução das instruções
- A análise é **independente do hardware** — vale para qualquer plataforma
- Só precisamos analisar as sequências possíveis (contáveis e finitas)
- Se a análise prova corretude, **não precisamos refazê-la** ao mudar de máquina

Esta abstração permite **verificação formal** de programas concorrentes — provar que o programa está correto para todos os entrelaçamentos possíveis. Ferramentas: Spin (spinroot.com)

REPETIBILIDADE E DEPURAÇÃO

Por causa do entrelaçamento arbitrário, é difícil ou impossível repetir precisamente uma execução.

O QUE NÃO FUNCIONA

1. Executar o programa
2. Diagnosticar o problema
3. Corrigir o código
4. Executar novamente para verificar

⚠ A nova execução pode seguir um entrelaçamento diferente — o bug pode desaparecer ou outro pode surgir.

"HEISENBUG"

Bug que desaparece quando se tenta observá-lo:

- Modo de depuração muda o timing
- Imprimir mensagens altera o entrelaçamento
- Instrumentação muda o comportamento

"The act of observing a system inevitably alters its state." — Werner Heisenberg

EXERCÍCIO

Considere o programa a seguir usando a notação de concorrência utilizada na disciplina:

boolean flag, turn ← false	
p	**q**
p1: **while not** flag p2: turn ← **not** turn	q1: **while not** flag q2: **if not** turn q3: flag ← true

1. Construa a **representação tabular** de dois cenários de execução do programa: um em que o valor de `turn` termina em `true` e outro em `false`.
2. Existe algum cenário em que o programa **nunca termine**? Justifique sua resposta.

INSTRUÇÕES ATÔMICAS

Nível de atomicidade e sua importância para a corretude

INSTRUÇÕES ATÔMICAS

Uma instrução **atômica** é executada completamente, sem a possibilidade de entrelaçamento de outras instruções durante sua execução.

O nível de atomicidade depende da plataforma:

LINGUAGEM DE MÁQUINA

Uma instrução de máquina é tipicamente atômica

LINGUAGEM DE ALTO NÍVEL

Uma linha pode gerar **múltiplas** instruções de máquina

BIBLIOTECAS

Uma chamada de função pode envolver muitas operações

A corretude de um programa concorrente depende da especificação do nível de atomicidade.

EXEMPLO 1 — INSTRUÇÕES ATÔMICAS

Algoritmo 2.3	
integer n ← 0	
p	**q**
p1: n ← n + 1	q1: n ← n + 1

Se `n←n+1` **é atômico:** apenas dois cenários possíveis. Em ambos, o valor final é **n = 2**.

> Algoritmo **correto** em relação à condição final n = 2.

p	q	n
p1: n←n+1	q1: n←n+1	0
(<end>)	**q1: n←n+1**	1
(<end>)	(<end>)	**2**

p	q	n
p1: n←n+1	**q1: n←n+1**	0
p1: n←n+1	(<end>)	1
(<end>)	(<end>)	**2**

EXEMPLO 2 — INSTRUÇÕES NÃO ATÔMICAS

Algoritmo 2.4: n ← n+1 decomposto em duas instruções	
integer n ← 0	
p	**q**
integer temp p1: temp ← n p2: n ← temp + 1	integer temp q1: temp ← n q2: n ← temp + 1

Cenário p1 → p2 → q1 → q2: n = 2 ✓

p	q	n	p.tmp	q.tmp
p1	q1	0	?	?
p2	q1	0	0	?
(<end>)	**q1**	1	0	?
(<end>)	**q2**	1	0	1
(<end>)	(<end>)	**2**	0	1

Cenário p1 → q1 → p2 → q2: n = 1 ✗

p	q	n	p.tmp	q.tmp
p1	q1	0	?	?
p2	**q1**	0	0	?
p2	q2	0	0	0
(<end>)	**q2**	1	0	0
(<end>)	(<end>)	**1**	0	0

ANÁLISE — NÍVEL DE ATOMICIDADE

O QUE MUDOU?

- A **mesma** operação lógica (`n←n+1`)
- Nível de atomicidade **diferente**
- No Ex. 1: atômica (1 instrução)
- No Ex. 2: não atômica (2 instruções)

CONSEQUÊNCIA

- Ex. 2 requer que `p2` execute logo após `p1`
- Mas o entrelaçamento **não garante** isso
- O algoritmo é **incorreto** para esse nível de atomicidade

A corretude de um programa concorrente depende da especificação do nível de atomicidade. Na prática, em linguagens de alto nível, operações como `n = n + 1` **não são atômicas**.

Em nível de máquina, o Ex. 1 **também pode ser decomposto** — na nossa abstração, instruções numeradas são **sempre atômicas** por definição.

CONDIÇÃO DE CORRIDA E SEÇÃO CRÍTICA

Problemas fundamentais da concorrência

CONDIÇÃO DE CORRIDA (RACE CONDITION / RACE HAZARD)

> Resultados inesperados causados por entrelaçamentos problemáticos. O comportamento depende da **sequência ou tempo de execução** de eventos fora do nosso controle.

- Pode se tornar um **erro** se um dos entrelaçamentos for indesejável
- Pode acontecer em diferentes níveis: circuitos, programas, sistemas distribuídos
- Termo introduzido por **David A. Huffman** (1954) — originalmente sobre circuitos digitais
- O problema é **difícil de reproduzir** por sua natureza não-determinística

DATA RACE

Um tipo particular de condição de corrida: dois processos concorrentes **acessam a mesma memória compartilhada** e pelo menos um deles realiza uma **escrita**.

PADRÃO DE DATA RACE

1. **Ler** o valor atual
2. **Alterar** localmente
3. **Escrever** de volta

Se outro processo interfere entre os passos, o resultado pode ser incorreto.

NEM TODA RACE CONDITION É DATA RACE

- Condição de corrida no acesso a um **recurso** (ex.: impressora)
- Condição de corrida no **envio de mensagens**
- Data race é especificamente sobre **memória compartilhada**

O PROBLEMA DA SEÇÃO CRÍTICA

Programas com entrelaçamentos que interrompem a execução em **regiões críticas** e resultam em erro:

EXEMPLOS

- Atualização do saldo de conta bancária
- Contador global compartilhado
- Impressão formatada na tela

ESTRUTURA DO PROBLEMA (ALGORITMO 3.1)

Algoritmo 3.1: Problema da seção crítica

p	q
`loop forever` seção não-crítica *preprotocol* **seção crítica** *postprotocol*	`loop forever` seção não-crítica *preprotocol* **seção crítica** *postprotocol*

Solução: garantir execução **atômica** da região crítica via **mecanismos de sincronização**.

REQUISITOS PARA A SOLUÇÃO DA SEÇÃO CRÍTICA

1. **Exclusão mútua:** apenas um processo estará em sua seção crítica de cada vez.
2. **Ausência de deadlock:** se processos querem entrar, uns não podem bloquear os outros indefinidamente.
3. **Ausência de starvation:** nenhum processo ficará indefinidamente esperando para entrar.

A solução deve funcionar para **qualquer número de processos** compartilhando o mesmo recurso.

PROPRIEDADES E PROBLEMAS

Safety, liveness, fairness e problemas clássicos

PROPRIEDADES DE CORRETUDE

Uma **propriedade** de um sistema é um atributo que é verdadeiro para **cada execução possível** do sistema. Toda propriedade pode ser formulada em termos de duas classes:

SAFETY (SEGURANÇA)

"Nada de ruim acontece."

Garantia de que o sistema **nunca** entra em um estado indesejável.

- Exclusão mútua
- Ausência de deadlock
- Correção parcial

LIVENESS (VIVACIDADE)

"Algo de bom eventualmente acontece."

Garantia de que o sistema **eventualmente** realiza uma ação desejada.

- Ausência de starvation
- Ausência de livelock
- Terminação

SAFETY E LIVENESS EM SISTEMAS CONCORRENTES

Propriedade	Tipo	Descrição
Exclusão mútua	Safety	No máximo um processo na seção crítica
Ausência de deadlock	Safety	O sistema nunca entra em bloqueio mútuo
Correção parcial	Safety	Se termina, o estado é correto
Ausência de starvation	Liveness	Nenhum processo é indefinidamente preterido
Ausência de livelock	Liveness	Processos não ficam em coordenação inútil
Terminação	Liveness	O programa eventualmente termina

Correção total = safety + liveness: o programa termina **e** o estado final é correto.

JUSTIÇA (FAIRNESS)

O escalonador garante que processos aptos **eventualmente executam**?

INCONDICIONAL

Processo apto **sempre** executa eventualmente.

FRACA

Executa se a condição permanece **continuamente** verdadeira.

FORTE

Executa se a condição é verdadeira **infinitas vezes** (mesmo que oscile).

> Forte ⇒ fraca ⇒ incondicional. O **escalonador do SO/runtime** determina o nível.

FAIRNESS — EXEMPLOS

INCONDICIONAL	Escalonador **round-robin** — cada processo roda por um quantum. Sem condições externas, ambos progridem.
FRACA	Semáforo simples (sem fila). Se o recurso fica **continuamente livre**, p entra. Se q aloca e libera repetidamente (condição **oscila**), p pode nunca entrar.
FORTE	Semáforo **FIFO**. Mesmo que q aloque e libere repetidamente (condição verdadeira **infinitas vezes**), p eventualmente entra — a fila garante a vez.

> Forte ⇒ fraca ⇒ incondicional. O **escalonador do SO** e os **mecanismos de sincronização** determinam o nível.

FAIRNESS — HIERARQUIA

Justiça forte ⇒ **Justiça fraca** ⇒ **Justiça incondicional**

Justiça forte **implica** fraca, que **implica** incondicional — mas não o inverso.

QUEM DETERMINA O NÍVEL?

- O **escalonador** do SO/runtime
- Os **mecanismos de sincronização** usados
- A **implementação** da linguagem

NA PRÁTICA

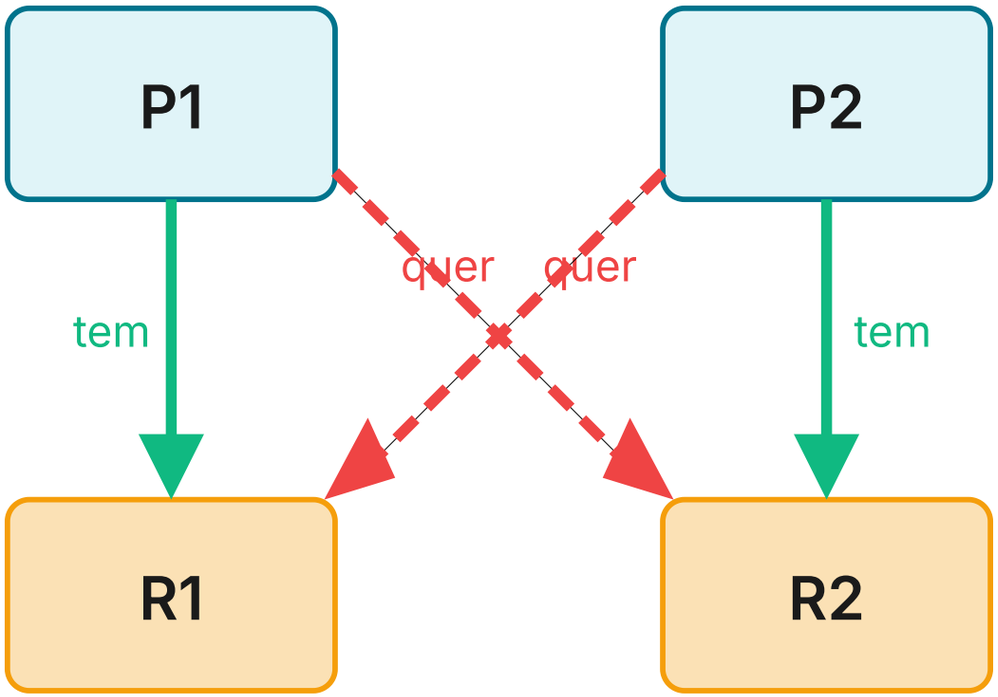
- A maioria dos SOs garante justiça **fraca**
- Justiça forte requer **semáforos FIFO**, aging, etc.
- Starvation freedom requer pelo menos justiça **fraca**

DEADLOCK

Situação onde dois ou mais processos esperam **indefinidamente** porque cada um está esperando por um recurso alocado por outro processo do grupo (**espera circular**).

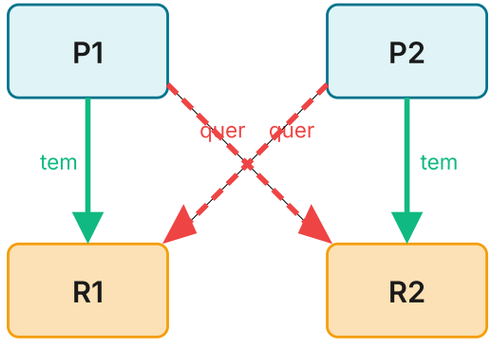
EXEMPLOS

- Alocação de recursos em sequência diferente (A → B vs. B → A)
- Problemas de protocolo: "só entro depois que todos entrarem"



DEADLOCK EM GO

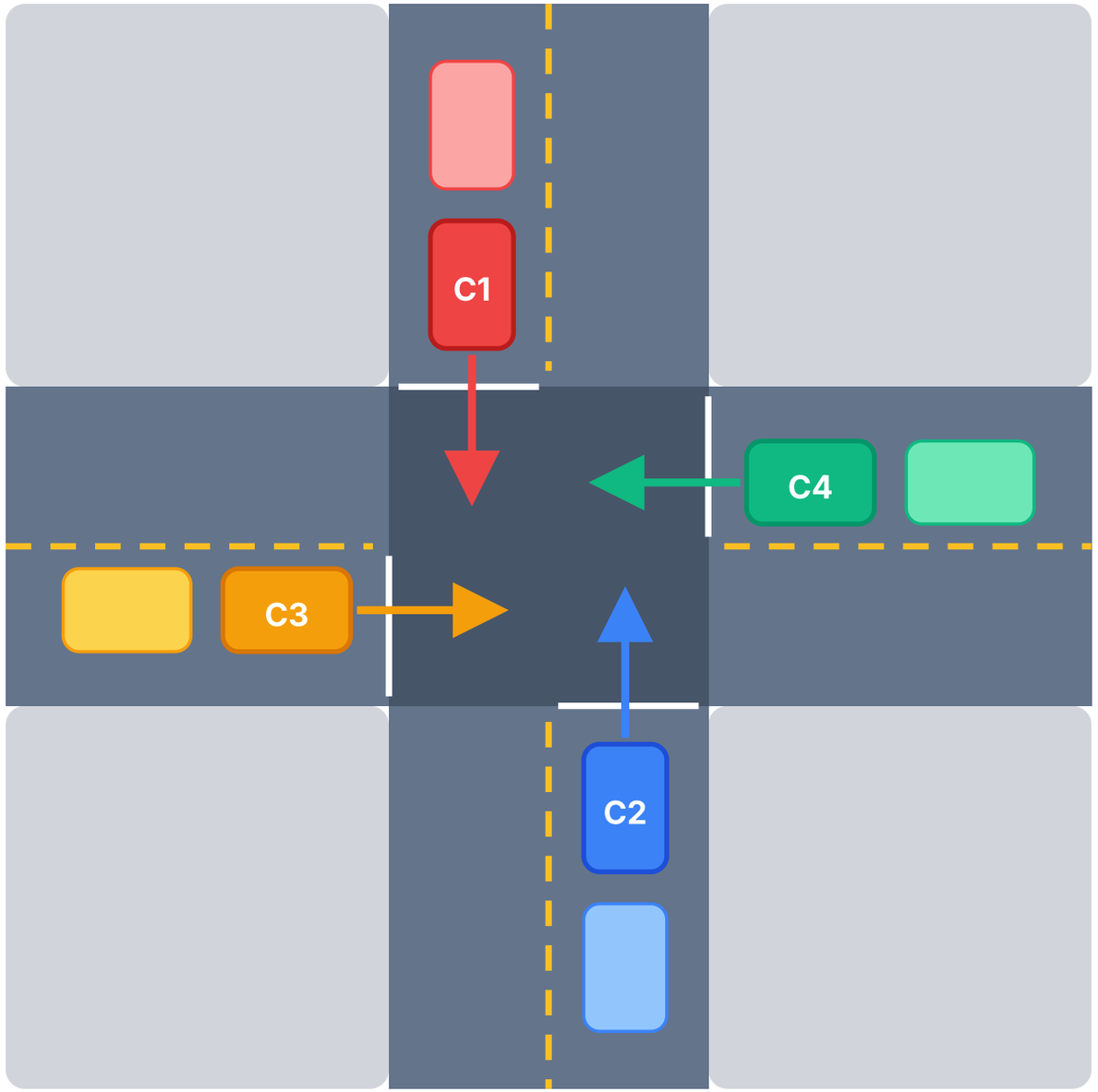
```go
var R1, R2 sync.Mutex

func P1() {
    R1.Lock()        // aloca R1
    time.Sleep(time.Millisecond)
    R2.Lock()        // espera R2... bloqueado!
    R1.Unlock()
    R2.Unlock()
}

func P2() {
    R2.Lock()        // aloca R2
    time.Sleep(time.Millisecond)
    R1.Lock()        // espera R1... bloqueado!
    R2.Unlock()
    R1.Unlock()
}

func main() {
    go P1()
    go P2()
    time.Sleep(5 * time.Second)
}
```



$ go run -race deadlock.go

fatal error: all goroutines are asleep - deadlock!

CRUZAMENTO DE QUATRO VIAS



- **C1** ocupa a via norte e espera **C3** passar
- **C3** ocupa a via oeste e espera **C2** passar
- **C2** ocupa a via sul e espera **C4** passar
- **C4** ocupa a via leste e espera **C1** passar

> Cada carro **segura** sua via (hold) e **espera** a via do próximo (wait) — formando espera circular. As quatro condições de Coffman estão satisfeitas.

DEADLOCK NA VIDA REAL

Todas as quatro condições de Coffman satisfeitas — nenhum carro consegue avançar.

CONDIÇÕES PARA DEADLOCK (COFFMAN, 1971)

Deadlock ocorre se e somente se **todas** as condições estão presentes:

1. **Exclusão mútua:** recursos não são compartilháveis simultaneamente.
2. **Hold and wait:** um processo pode solicitar novos recursos enquanto mantém outros alocados.
3. **Não preempção:** recursos não podem ser retirados à força de processos que os alocaram.
4. **Espera circular:** forma-se um ciclo de dependências entre processos por recursos.

> Podemos evitar deadlock **eliminando qualquer uma** dessas condições.

LIVELOCK

Processos **executam continuamente**, mas realizam apenas ações de coordenação — **nenhum trabalho útil** é feito.

CARACTERÍSTICAS

- **Diferente de deadlock:** os processos não estão bloqueados — estão ativos
- Ciclo infinito de **tentativa** → **conflito** → **recuo** → **tentativa**
- Geralmente ocorre quando processos usam a **mesma estratégia** para evitar conflitos

EXEMPLOS

- Duas pessoas num corredor movendo-se para o mesmo lado
- Dois processos liberam e readquirem os mesmos recursos em sincronia
- Protocolos de recuo simétrico em redes

Solução: introduzir **aleatoriedade** ou **assimetria** na estratégia de recuo — como no protocolo Ethernet (CSMA/CD com backoff exponencial aleatório).

PROTOCOLO DE POLIDEZ INFINITA

Via, resti servita Madama brillante
– E. Tommasi Ferroni, 2012

Ambas gesticulam, cedem passagem, tentam se mover — mas repetem o mesmo ciclo indefinidamente.

Há ação, mas não há progresso.

STARVATION

Um processo está apto a executar, mas é **indefinidamente preterido** — o sistema funciona, mas ele nunca consegue sua vez.

CARACTERÍSTICAS

- O sistema como um todo **progride normalmente**
- Apenas um ou alguns processos são prejudicados
- Causada por **políticas de escalonamento injustas** (ex.: prioridade fixa)

EXEMPLOS

- Impressora sempre atende trabalhos prioritários — trabalho pequeno nunca imprime
- Elevador prioriza andares próximos — andar distante nunca é atendido
- Semáforo fraco: processo liberado é escolhido arbitrariamente

Solução: garantir **justiça** (fairness) — filas FIFO, aging (aumento gradual de prioridade) ou semáforos fortes.

DEADLOCK VS. LIVELOCK VS. STARVATION

	Deadlock	Livelock	Starvation
Processos bloqueados?	Sim	Não (executam)	Um ou mais
Sistema progride?	Não	Não (sem trabalho útil)	Sim (exceto o preterido)
Abrangência	Sistema inteiro	Sistema inteiro	Processo individual
Analogia	Carros bloqueados num cruzamento	Pessoas no corredor movendo-se para o mesmo lado	Cliente na fila que nunca é atendido

Os três são problemas de **liveness**. Deadlock e livelock afetam o sistema inteiro; starvation afeta processos individuais enquanto o restante funciona normalmente.

RESUMO

- Programa concorrente = processos sequenciais com entrelaçamento arbitrário de instruções atômicas
- O nível de atomicidade determina a corretude — operações de alto nível geralmente não são atômicas
- Condição de corrida e data race: resultados dependem do entrelaçamento
- Safety ("nada ruim") e liveness ("algo bom eventualmente") são as propriedades fundamentais
- Deadlock, livelock e starvation são problemas centrais da concorrência